# Programming Language Graph — v2 Roadmap Plan (Modified with Research Audit)

This document contains the modified v2 roadmap for the Programming Language Graph project. The roadmap preserves the original phase structure while integrating insights from the deep research audit and UX recommendations for graph visualization systems.

## Phase 0 — Foundation Audit

Goal: Stabilize dataset integrity and prepare the system for scaling.

- Audit dataset edges and verify all language references.
- Remove dangling edges and duplicated relationships.
- Validate schema using Zod with strict type checks.
- Ensure year ranges and confidence scores are valid.
- Add automated checks for duplicate nodes, circular bootstrap chains, and orphan languages.
- Generate dataset statistics via scripts/analyzeDataset.ts.
- Compute graph metadata such as in-degree, out-degree, and connected components.
- Validate historical logic for compiler and bootstrap relationships.

## Phase 1 — Dataset Expansion

Goal: Enrich historical depth and contextual metadata.

- Add additional languages including BCPL, B, Pascal, Ada, Lisp, Scheme, Smalltalk, Objective■C, Swift, Kotlin, Zig, Nim, Crystal, Haskell.
- Introduce additional edge types including influenced_by and inspired_by.
- Extend language schema with designer, company, paradigm, typing model, garbage collection, and logo URLs.
- Add edge metadata including start_year, end_year, notes, and evidence URLs.
- Include popularity metrics such as current_users, peak_users, and peak_year.
- Attach official language logos for improved recognition.

## Phase 2 — Visual Graph Improvements

Goal: Improve readability and exploration clarity.

- Edge styling based on confidence levels and relationship types.
- Dashed edges for bootstrap relationships.
- Arrowheads to show direction of implementation.

- Node colors grouped by language categories.
- Node size based on degree centrality.
- Additional layouts such as lineage, timeline, and cluster layouts.
- Node hover highlighting with neighbor emphasis.
- Contextual right■click menus for graph exploration actions.

## Phase 3 — Timeline Visualization

Goal: Show historical evolution of programming languages.

- Horizontal timeline axis with nodes positioned by release year.
- Interactive year slider controlling visible languages.
- Animated evolution showing the growth of the graph over time.
- Decade filters to explore historical periods.

## Phase 4 — Bootstrap Chain Explorer

Goal: Highlight compiler lineage and self■hosting transitions.

- DFS traversal to compute bootstrap chains.
- Interactive path highlighting within the graph.
- Display metadata including bootstrap years and transitions to self■hosting.

## Phase 5 — Influence Graph

Goal: Visualize conceptual influence between languages.

- Add influenced_by edges supported by documented sources.
- Provide toggle between implementation graph and influence graph modes.
- Maintain evidence URLs for every influence relationship.

## Phase 6 — Advanced Exploration Tools

Goal: Enable deeper exploration and analytics.

- Graph analytics such as PageRank and centrality rankings.
- Filtering by paradigm and language category.
- Improved search supporting partial matches.
- Relationship■type filtering.
- Display graph statistics including cluster size and bootstrap depth.

## Phase 7 — Graph Insights Panel

Goal: Transform the graph into an analytical knowledge tool.

- Display key insights such as longest bootstrap chain.
- Identify most influential and self■hosting languages.
- Show statistics derived from graph analytics.

## Phase 8 — Story Mode

Goal: Provide guided narrative exploration.

- Preset stories such as the C lineage and VM ecosystems.
- Step■through interaction highlighting nodes sequentially.
- Interactive story navigation with next/previous steps.

## Phase 9 — Performance Optimization

Goal: Ensure smooth performance as dataset grows.

- Precompute node coordinates and store preset layouts.
- Use cy.batch() for bulk Cytoscape updates.
- Reduce animation overhead and maintain simple node shapes.
- Implement clustering and level■of■detail rendering.

## Phase 10 — Production Polish

Goal: Deliver a portfolio■grade interactive system.

- Add dark mode and language logos.
- Implement node hover cards with detailed metadata.
- Enable graph export to SVG or PNG.
- Provide shareable URLs encoding graph state.
- Improve accessibility with keyboard navigation and ARIA labels.
- Enhance mobile usability and responsive controls.
- Update README with architecture, dataset schema, and deployment workflow.

## Final v2 Vision

- Transform the project from a static exploration graph into an interactive programming language atlas.
- Support timeline evolution, bootstrap chain visualization, influence graphs, analytics insights, and guided stories.